

Nivel 9: directorios.c {mi_write(), mi_read()}, y mi_escribir.c, mi_cat.c

comandos:	directorios.c:	ficheros.c:
mi_mkdir, [mi_touch] mi_ls mi_chmod mi_stat mi_escribir mi_cat mi_link mi_rm, [mi_rmdir], [mi_rm_r]	mi_creat() mi_dir() mi_chmod() mi_stat() mi_write() mi_read() mi_link() mi_unlink()	 mi_chmod_f() mi_stat_f() mi_write_f() mi_read_f() mi_truncar_f()

Tabla con la correlación de comandos y funciones de la capa de directorios y la capa de ficheros

5) Escritura en un offset de un fichero

5a) mi_escribir.c

Sintaxis: ./mi_escribir <disco> </ruta_fichero> <texto> <offset>

Permite escribir texto en una posición de un fichero (*offset*). Podéis adaptar [escribir.c](#) de la entrega parcial de la siguiente manera:

- Ha de incluir [directorios.h](#) en vez de [ficheros.h](#).
- Ha de recibir como parámetro la **ruta del fichero** en vez del **nº de inodo** (y ya no hay que llamar a [reservar_inodo\(\)](#) de forma artificial, ya que eso lo hará de forma natural [buscar_entrada\(\)](#)).
- Eliminar el argumento de [diferentes_inodos](#) y añadir el de [offset](#) para poder indicar **desde consola** dónde vamos a escribir¹.
- En vez de llamar a [mi_write_f\(\)](#) ha de llamar a [mi_write\(\)](#).

¹ Por tanto **ya no hay que escribir en diferentes offsets prefijados** sino solo en el que nos indiquen desde consola.

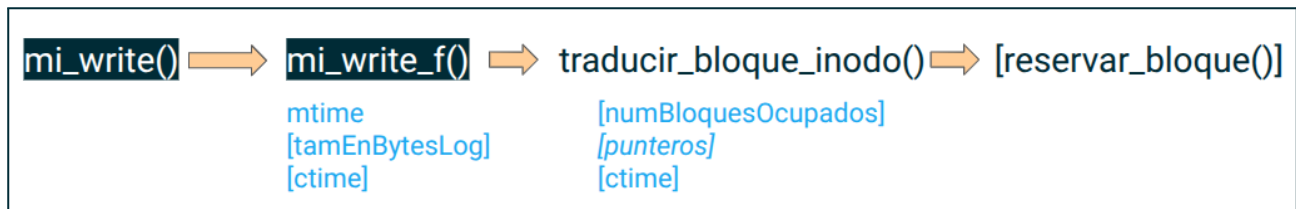
`mi_write_f()` de la capa de ficheros, que será llamada por `mi_write()` de la capa de directorios ya comprobará que se trate de un **fichero**, y que tenga **permisos de escritura**.

Mostrar la cantidad de **bytes escritos**.

5b) `int mi_write(const char *camino, const void *buf, unsigned int offset, unsigned int nbytes);`

Función de `directorios.c` para escribir contenido en un fichero. Buscaremos la entrada `camino` con `buscar_entrada()` para obtener el `p_inodo`. Si la entrada existe llamamos a la función correspondiente de `ficheros.c` pasándole el `p_inodo`:

```
mi_write_f(p_inodo, buf, offset, nbytes);
```



Principales funciones de capas inferiores y campos del inodo involucrados

Ha de devolver los **bytes escritos**.

Observaciones:

- Se puede **mejorar** la operación de escritura utilizando una **caché de directorios**: guardando el camino de la **última entrada buscada** y su `p_inodo` correspondiente (o un array de las **n últimas entradas**), dado que la mayoría de las lecturas/escrituras seguidas se suelen hacer sobre el mismo inodo ("**principio de proximidad**"), de esta forma, garantizaríamos que solo se busque la entrada si no coincide con la/s última/s buscada/s.
 - Definiríamos, en `directorios.c`, un `struct` con el **camino** asociado a un **nº de inodo**:

```
struct UltimaEntrada{  
    char camino [TAMNOMBRE*PROFUNDIDAD];  
    int p_inodo;  
};
```

donde

```
#define TAMNOMBRE 60 //tamaño del nombre de directorio o fichero  
#define PROFUNDIDAD 32 //profundidad máxima del árbol de directorios
```

- Y una variable global que guarde la última entrada para escritura:

```
static struct UltimaEntrada UltimaEntradaEscritura;
```

- Comprobaríamos si la escritura es sobre el mismo inodo:

```
strcmp(UltimaEntradaEscritura.camino, camino) == 0
```

- Si lo es, entonces:

```
p_inodo = UltimaEntradaEscritura.p_inodo;
```

- si no llamar a `buscar_entrada()` y actualizar los campos de `UltimaEntradaEscritura` con el `p_inodo` obtenido para el `camino` buscado.

También podéis utilizar un array de registros tipo `UltimaEntrada`, en el que la posición 0 sea por ejemplo para escritura y la posición 1 para lectura.

O incluso tener una **caché de más profundidad** utilizando un **array de n últimas entradas** de lectura o de escritura (por ejemplo $n=3$). Para la gestión de esta tabla podéis probar diferentes estrategias: **FIFO**, **LRU**, ...².

- En el caso de **FIFO**, para no desplazar todos los elementos cuando se inserte uno nuevo estando la tabla llena, se puede guardar la posición de la última inserción y tratar la tabla de manera **circular** mediante el operador **módulo**.
- En el caso de **LRU** se podría grabar, para cada entrada de la tabla, un **sello de tiempo** en **microsegundos** usando el `struct timeval` y la función `gettimeofday()`³.
 - Si aplicamos esta estrategia habría que modificar la declaración de la última entrada, por ejemplo de la siguiente manera:

```
#define USARCACHE 3 //0:sin caché, 1: última L/E, 2:tabla FIFO, 3:tabla LRU

struct UltimaEntrada{
    char camino [TAMNOMBRE*PROFUNDIDAD];
    int p_inodo;
    #if USARCACHE==3 // tabla LRU
        struct timeval ultima_consulta;
    #endif
};

//tabla caché directorios
#if (USARCACHE==2 || USARCACHE==3)
    #define CACHE_SIZE 3 // cantidad de entradas para la caché
    static struct UltimaEntrada UltimasEntradas[CACHE_SIZE];
#endif
```

² Ver **algoritmos de reemplazo de paginación** para memoria principal.

³ El primer argumento es nuestra variable de tipo `struct timeval` y el segundo será `NULL`.

6) Lectura secuencial de todo el contenido de un fichero

6a) mi_cat.c

Sintaxis: ./mi_cat <disco> </ruta_fichero>

Programa (comando) que muestra **TODO** el contenido de un fichero. Podéis adaptar [leer.c](#) de la entrega parcial para que reciba como parámetro la **ruta del fichero** en vez del **nº de inodo** (incluyendo ahora [directorios.h](#) en vez de [ficheros.h](#)) y ha de llamar a [mi_read\(\)](#) en vez de a [mi_read_f\(\)](#).

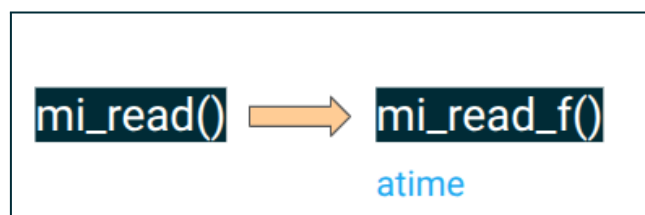
Observaciones:

- Hay que comprobar que la ruta se corresponda a un fichero ya que si es un directorio no podemos hacer un cat.
- Tendrán que coincidir los [bytes leídos](#) con el [tamaño en bytes lógico](#) del fichero y con el [tamaño físico del fichero externo](#) al que **redireccionemos** la lectura, y se ha de **filtrar** la basura.
- Utilizar una variable por ej. [tambuffer](#) para poder cambiar el **tamaño del buffer de lectura** sin tener que modificar todas las sentencias involucradas. Recordad probar con diferentes tamaños, tanto múltiplos de varios bloques como con un valor cualquiera, como por ejemplo 1500 bytes.

6b) `int mi_read(const char *camino, void *buf, unsigned int offset, unsigned int nbytes);`

Función de [directorios.c](#) para leer los [nbytes](#) del fichero indicado por [camino](#), a partir del [offset](#) pasado por parámetro y copiarlos en el buffer [buf](#). Buscaremos la entrada [camino](#) con [buscar_entrada\(\)](#) para obtener el [p_inodo](#). Si la entrada existe llamamos a la función correspondiente de [ficheros.c](#) pasándole el [p_inodo](#):

```
mi_read_f(p_inodo, buf, offset, nbytes);
```



Principales funciones de capas inferiores y campos del inodo involucrados

Ha de devolver los **bytes leídos**.

También se puede **optimizar** utilizando una **caché de directorios** guardando el **camino** y **p_inodo** de la **última lectura** o de las **n últimas lecturas**.

TESTS DE PRUEBA⁴

Podéis ejecutar línea a línea⁵ o usar el script **test9.sh** (lo adaptáis si es preciso, éste utiliza **mi_touch**, **mi_ls -l**, y llama a **mi_escribir_varios** para **comprobar la caché de directorios que guarda 1 entrada para escritura y otra para lectura**, siendo el tamaño del buffer de lectura, **tambuffer**, de 4KBs).

Para probar una caché tipo **tabla con FIFO o LRU** hay que utilizar el programa **prueba_cache_tabla.c** y mostrar cuando se actualiza la caché y cuando se utiliza (ver Anexo).

```
uib:~$ ./test9.sh
#####
$ ./mi_mkfs disco 100000

#####
$ ./mi_touch disco 6 /fichero

$ ./mi_ls -l disco /
Total: 1
Tipo  Modo mTime                Tamaño                Nombre
-----
f      rw-   2025-02-12 15:10:29          0                    fichero

$ ./mi_escribir disco /fichero hola que tal 5120
longitud texto: 12
[mi_write() → Actualizamos la caché de escritura]
Bytes escritos: 12

$ ./mi_ls -l disco /
Total: 1
Tipo  Modo mTime                Tamaño                Nombre
-----
f      rw-   2025-02-12 15:10:29        5132                    fichero

$ ./leer_sf disco
```

⁴ Aquí ya se pueden omitir los **fprintf()** de **buscar_entrada()** salvo los errores y dejar comentadas las llamadas a **buscar_entrada()** de **leer_sf.c**

⁵ Ejecutado línea a línea manualmente observaréis que varían los sellos de tiempo entre sí.

DATOS DEL SUPERBLOQUE

posPrimerBloqueMB = 1
posUltimoBloqueMB = 13
posPrimerBloqueAI = 14
posUltimoBloqueAI = 3138
posPrimerBloqueDatos = 3139
posUltimoBloqueDatos = 99999
posInodoRaiz = 0
posPrimerInodoLibre = 2
cantBloquesLibres = 96859
cantInodosLibres = 24998
totBloques = 100000
totInodos = 25000

\$./mi_chmod disco 4 /fichero

\$./mi_escribir disco /fichero estoy estupendamente 256000

longitud texto: 20

[mi_write() → Actualizamos la caché de escritura]

No hay permisos de escritura

Bytes escritos: 0

\$./mi_ls -l disco /fichero #si no está implementado mi_ls para ficheros toca dar error

Tipo	Modo	mTime	Tamaño	Nombre
f	r--	2025-02-12 15:10:29	5132	fichero

\$./mi_mkdir disco 6 /dir1/

\$./mi_touch disco 6 /dir1/fic1

./mi_escribir disco /dir1/fic1 hola1 256000

longitud texto: 5

[mi_write() → Actualizamos la caché de escritura]

Bytes escritos: 5

\$./mi_stat disco /dir1/fic1

Nº de inodo: 3

tipo: f

permisos: 6

atime: Wed 2025-02-12 15:10:29

mtime: Wed 2025-02-12 15:10:29

```
ctime: Wed 2025-02-12 15:10:29
btime: Wed 2025-02-12 15:10:29
nlinks: 1
tamEnBytesLog: 256005
numBloquesOcupados: 2
```

```
#####
$ sleep 1 #esperamos un poco para observar los sellos de tiempo
```

```
#####
$ ./mi_escribir disco /dir1/fic1 hola2 5120 #no cambia tamenBytesLog pero sí mtime y
ctime (ocupamos 1 bloque más)
longitud texto: 5
[mi_write() → Actualizamos la caché de escritura]
Bytes escritos: 5
```

```
$ ./mi_stat disco /dir1/fic1
Nº de inodo: 3
tipo: f
permisos: 6
atime: Wed 2025-02-12 15:10:29
mtime: Wed 2025-02-12 15:10:30
ctime: Wed 2025-02-12 15:10:30
btime: Wed 2025-02-12 15:10:29
nlinks: 1
tamEnBytesLog: 256005
numBloquesOcupados: 3
```

```
#####
$ sleep 1 #esperamos un poco para observar los sellos de tiempo
```

```
#####
$ ./mi_escribir disco /dir1/fic1 hola3 5200 #mismo bloque que offset 5120, cambia
mtime pero no ctime
longitud texto: 5
[mi_write() → Actualizamos la caché de escritura]
Bytes escritos: 5
```

```
$ ./mi_stat disco /dir1/fic1
Nº de inodo: 3
tipo: f
permisos: 6
atime: Wed 2025-02-12 15:10:29
mtime: Wed 2025-02-12 15:10:31
ctime: Wed 2025-02-12 15:10:30
```

btime: Wed 2025-02-12 15:10:29

nlinks: 1

tamEnBytesLog: 256005

numBloquesOcupados: 3

#####

sleep 1 #esperamos un poco para observar los sellos de tiempo

#####

./mi_escribir disco /dir1/fic1 hola4 256010 #cambia tamEnBytesLog, mtime y ctime

longitud texto: 5

[mi_write() → Actualizamos la caché de escritura]

Bytes escritos: 5

\$./mi_stat disco /dir1/fic1

Nº de inodo: 3

tipo: f

permisos: 6

atime: Wed 2025-02-12 15:10:29

mtime: Wed 2025-02-12 15:10:32

ctime: Wed 2025-02-12 15:10:32

btime: Wed 2025-02-12 15:10:29

nlinks: 1

tamEnBytesLog: 256015

numBloquesOcupados: 3

#####

Comprobación de la caché de directorios (1 entrada para L y otra para E)

#####

\$./mi_touch disco 6 /dir1/fic2

\$./mi_escribir disco /dir1/fic2 ¿Qué es Lorem Ipsum? Lorem Ipsum es simplemente el texto de relleno de las imprentas y archivos de texto. Lorem Ipsum ha sido el texto de

...

repeticiones, humor agregado o palabras no características del lenguaje, etc. 1000

longitud texto: 3751

[mi_write() → Actualizamos la caché de escritura]

Bytes escritos: 3751

#####

\$./mi_cat disco /dir1/fic2 #tambuffer=BLOCKSIZE * 4

[mi_read() → Actualizamos la caché de lectura]

¿Qué es Lorem Ipsum?

Lorem Ipsum es simplemente el texto de relleno de las imprentas y archivos de texto. Lorem Ipsum ha sido el texto de relleno estándar de las industrias desde el año 1500, cuando un impresor (N. del T. persona que se dedica a la imprenta) desconocido usó una galería de textos y los mezcló de tal manera que logró hacer un libro de textos especimen. No sólo sobrevivió 500 años, sino que tambien ingresó como texto de relleno en documentos electrónicos, quedando esencialmente igual al original. Fue popularizado en los 60s con la creación de las hojas "Letraset", las cuales contenian pasajes de Lorem Ipsum, y más recientemente con software de autoedición, como por ejemplo Aldus PageMaker, el cual incluye versiones de Lorem Ipsum.

¿Por qué lo usamos?

Es un hecho establecido hace demasiado tiempo que un lector se distraerá con el contenido del texto de un sitio mientras que mira su diseño. El punto de usar Lorem Ipsum es que tiene una distribución más o menos normal de las letras, al contrario de usar textos como por ejemplo "Contenido aquí, contenido aquí". Estos textos hacen parecerlo un español que se puede leer. Muchos paquetes de autoedición y editores de páginas web usan el Lorem Ipsum como su texto por defecto, y al hacer una búsqueda de "Lorem Ipsum" va a dar por resultado muchos sitios web que usan este texto si se encuentran en estado de desarrollo. Muchas versiones han evolucionado a través de los años, algunas veces por accidente, otras veces a propósito (por ejemplo insertándole humor y cosas por el estilo).

¿De dónde viene?

Al contrario del pensamiento popular, el texto de Lorem Ipsum no es simplemente texto aleatorio. Tiene sus raíces en una pieza clásica de la literatura del Latin, que data del año 45 antes de Cristo, haciendo que este adquiera mas de 2000 años de antigüedad. Richard McClintock, un profesor de Latin de la Universidad de Hampden-Sydney en Virginia, encontró una de las palabras más oscuras de la lengua del latín, "consecteur", en un pasaje de Lorem Ipsum, y al seguir leyendo distintos textos del latín, descubrió la fuente indudable. Lorem Ipsum viene de las secciones 1.10.32 y 1.10.33 de "de Finibus Bonorum et Malorum" (Los Extremos del Bien y El Mal) por Cicero, escrito en el año 45 antes de Cristo. Este libro es un tratado de teoría de éticas, muy popular durante el Renacimiento. La primera línea del Lorem Ipsum, "Lorem ipsum dolor sit amet..", viene de una línea en la sección 1.10.32

El trozo de texto estándar de Lorem Ipsum usado desde el año 1500 es reproducido debajo para aquellos interesados. Las secciones 1.10.32 y 1.10.33 de "de Finibus Bonorum et Malorum" por Cicero son también reproducidas en su forma original exacta, acompañadas por versiones en Inglés de la traducción realizada en 1914 por H. Rackham.

¿Dónde puedo conseguirlo?

Hay muchas variaciones de los pasajes de Lorem Ipsum disponibles, pero la mayoría sufrió alteraciones en alguna manera, ya sea porque se le agregó humor, o palabras

aleatorias que no parecen ni un

[mi_read() → Utilizamos la caché de lectura en vez de llamar a buscar_entrada()]

poco creíbles. Si vas a utilizar un pasaje de Lorem Ipsum, necesitas estar seguro de que no hay nada avergonzante escondido en el medio del texto. Todos los generadores de Lorem Ipsum que se encuentran en Internet tienden a repetir trozos predefinidos cuando sea necesario, haciendo a este el único generador verdadero (válido) en la Internet. Usa un diccionario de mas de 200 palabras provenientes del latín, combinadas con estructuras muy útiles de sentencias, para generar texto de Lorem Ipsum que parezca razonable. Este Lorem Ipsum generado siempre estará libre de repeticiones, humor agregado o palabras no características del lenguaje, etc.

[mi_read() → Utilizamos la caché de lectura en vez de llamar a buscar_entrada()]

Total_leidos 4751

#####

\$./mi_escribir disco /dir1/fic2 "*****" 10000

longitud texto: 30

[mi_write() → Actualizamos la caché de escritura]

Bytes escritos: 30

\$./mi_ls -l disco /dir1/

Total: 2

Tipo	Modo	mTime	Tamaño	Nombre
f	rw-	2025-02-12 15:10:32	256015	fic1
f	rw-	2025-02-12 15:10:32	10030	fic2

\$./mi_cat disco /dir1/fic2 #tambuffer=BLOCKSIZE * 4

[mi_read() → Actualizamos la caché de lectura]

¿Qué es Lorem Ipsum?

Lorem Ipsum es simplemente el texto de relleno de las imprentas y archivos de texto. Lorem Ipsum ha sido el texto de relleno estándar de las industrias desde el año 1500, cuando un impresor (N. del T. persona que se dedica a la imprenta) desconocido usó una galería de textos y los mezcló de tal manera que logró hacer un libro de textos especimen. No sólo sobrevivió 500 años, sino que tambien ingresó como texto de relleno en documentos electrónicos, quedando esencialmente igual al original. Fue popularizado en los 60s con la creación de las hojas "Letraset", las cuales contenian pasajes de Lorem Ipsum, y más recientemente con software de autoedición, como por ejemplo Aldus PageMaker, el cual incluye versiones de Lorem Ipsum.

¿Por qué lo usamos?

Es un hecho establecido hace demasiado tiempo que un lector se distraerá con el contenido del texto de un sitio mientras que mira su diseño. El punto de usar Lorem

Ipsum es que tiene una distribución más o menos normal de las letras, al contrario de usar textos como por ejemplo "Contenido aquí, contenido aquí". Estos textos hacen parecerlo un español que se puede leer. Muchos paquetes de autoedición y editores de páginas web usan el Lorem Ipsum como su texto por defecto, y al hacer una búsqueda de "Lorem Ipsum" va a dar por resultado muchos sitios web que usan este texto si se encuentran en estado de desarrollo. Muchas versiones han evolucionado a través de los años, algunas veces por accidente, otras veces a propósito (por ejemplo insertándole humor y cosas por el estilo).

¿De dónde viene?

Al contrario del pensamiento popular, el texto de Lorem Ipsum no es simplemente texto aleatorio. Tiene sus raíces en una pieza clásica de la literatura del Latín, que data del año 45 antes de Cristo, haciendo que este adquiera mas de 2000 años de antigüedad. Richard McClintock, un profesor de Latín de la Universidad de Hampden-Sydney en Virginia, encontró una de las palabras más oscuras de la lengua del latín, "consectetur", en un pasaje de Lorem Ipsum, y al seguir leyendo distintos textos del latín, descubrió la fuente indudable. Lorem Ipsum viene de las secciones 1.10.32 y 1.10.33 de "de Finibus Bonorum et Malorum" (Los Extremos del Bien y El Mal) por Cicero, escrito en el año 45 antes de Cristo. Este libro es un tratado de teoría de éticas, muy popular durante el Renacimiento. La primera línea del Lorem Ipsum, "Lorem ipsum dolor sit amet..", viene de una línea en la sección 1.10.32

El trozo de texto estándar de Lorem Ipsum usado desde el año 1500 es reproducido debajo para aquellos interesados. Las secciones 1.10.32 y 1.10.33 de "de Finibus Bonorum et Malorum" por Cicero son también reproducidas en su forma original exacta, acompañadas por versiones en Inglés de la traducción realizada en 1914 por H. Rackham.

¿Dónde puedo conseguirlo?

Hay muchas variaciones de los pasajes de Lorem Ipsum disponibles, pero la mayoría sufrió alteraciones en alguna manera, ya sea porque se le agregó humor, o palabras aleatorias que no parecen ni un

[\[mi_read\(\) → Utilizamos la caché de lectura en vez de llamar a buscar_entrada\(\)\]](#)

poco creíbles. Si vas a utilizar un pasaje de Lorem Ipsum, necesitás estar seguro de que no hay nada avergonzante escondido en el medio del texto. Todos los generadores de Lorem Ipsum que se encuentran en Internet tienden a repetir trozos predefinidos cuando sea necesario, haciendo a este el único generador verdadero (válido) en la Internet. Usa un diccionario de mas de 200 palabras provenientes del latín, combinadas con estructuras muy útiles de sentencias, para generar texto de Lorem Ipsum que parezca razonable. Este Lorem Ipsum generado siempre estará libre de repeticiones, humor agregado o palabras no características del lenguaje, etc.

[\[mi_read\(\) → Utilizamos la caché de lectura en vez de llamar a buscar_entrada\(\)\]](#)

[\[mi_read\(\) → Utilizamos la caché de lectura en vez de llamar a buscar_entrada\(\)\]](#)

```
Total_leidos 10030
#####
$ ./mi_touch disco 6 /dir1/fic3

$ ./mi_escribir_varios disco /dir1/fic3 "--texto repetido en 10 bloques--" 0
longitud texto: 32
[mi_write() → Actualizamos la caché de escritura]
[mi_write() → Utilizamos la caché de escritura en vez de llamar a buscar_entrada()]
[mi_write() → Utilizamos la caché de escritura en vez de llamar a buscar_entrada()]
[mi_write() → Utilizamos la caché de escritura en vez de llamar a buscar_entrada()]
[mi_write() → Utilizamos la caché de escritura en vez de llamar a buscar_entrada()]
[mi_write() → Utilizamos la caché de escritura en vez de llamar a buscar_entrada()]
[mi_write() → Utilizamos la caché de escritura en vez de llamar a buscar_entrada()]
[mi_write() → Utilizamos la caché de escritura en vez de llamar a buscar_entrada()]
[mi_write() → Utilizamos la caché de escritura en vez de llamar a buscar_entrada()]
[mi_write() → Utilizamos la caché de escritura en vez de llamar a buscar_entrada()]
Bytes escritos: 320

$ ./mi_stat disco /dir1/fic3
Nº de inodo: 5
tipo: f
permisos: 6
atime: Wed 2025-02-12 15:10:32
mtime: Wed 2025-02-12 15:10:32
ctime: Wed 2025-02-12 15:10:32
btime: Wed 2025-02-12 15:10:32
nlinks: 1
tamEnBytesLog: 9248
numBloquesOcupados: 10

$ ./mi_cat disco /dir1/fic3 #tambuffer=BLOCKSIZE * 4
[mi_read() → Actualizamos la caché de lectura]
--texto repetido en 10 bloques---texto repetido en 10 bloques---texto repetido en 10
bloques---texto repetido en 10 bloques--
[mi_read() → Utilizamos la caché de lectura en vez de llamar a buscar_entrada()]
--texto repetido en 10 bloques---texto repetido en 10 bloques---texto repetido en 10
bloques---texto repetido en 10 bloques--
[mi_read() → Utilizamos la caché de lectura en vez de llamar a buscar_entrada()]
--texto repetido en 10 bloques---texto repetido en 10 bloques--
[mi_read() → Utilizamos la caché de lectura en vez de llamar a buscar_entrada()]

Total_leidos 9248
```


ANEXO

Programa para testear la caché de directorios con 1 última Lectura y 1 última Escritura

```
//mi_escribir_varios.c
//Programa para testear las cachés de directorios de 1 última L y 1 última E

#include "directorios.h"

int main(int argc, char **argv){

    //Comprobamos sintaxis
    if (argc!=5) {
        fprintf(stderr, "Sintaxis: mi_escribir_varios <nombre_dispositivo> </ruta_fichero>
        <texto> <offset>\n");
        exit(-1);
    }

    //montamos el dispositivo
    if(bmount(argv[1])<0) return -1;
    //obtenemos el texto y su longitud
    char *buffer_texto = argv[3];
    int longitud=strlen(buffer_texto);

    //obtenemos la ruta y comprobamos que no se refiera a un directorio
    if (argv[2][strlen(argv[2])-1]=='/') {
        fprintf(stderr, "Error: la ruta se corresponde a un directorio");
        exit(-1);
    }
    char *camino = argv[2];
    unsigned int offset=atoi(argv[4]); //obtenemos el offset
    int escritos=0;
    int varios = 10;
    fprintf(stderr, "longitud texto: %d\n", longitud);
    for (int i=0; i<varios; i++) {
        // escribimos varias veces el texto desplazado 1 bloque
        escritos += mi_write(camino,buffer_texto,offset+BLOCKSIZE*i,longitud);
    }
    fprintf(stderr, "Bytes escritos: %d\n", escritos);
    /* Visualización del stat
    struct STAT stat;
    mi_stat_f(ninodo, &stat);
    printf("stat.tamEnBytesLog=%d\n",stat.tamEnBytesLog);
```

```
printf("stat.numBloquesOcupados=%d\n",stat.numBloquesOcupados);
*/

bumount();
}
```

Ejemplo test para caché de directorios tamaño > 1 (TABLA)

Crea 5 ficheros: /fic1, /fic2, /fic3, /fic4 y /fic5, y luego genera escrituras en el siguiente orden de rutas:

/fic2, /fic3, /fic2, /fic1, /fic5, /fic2, /fic4, /fic5, /fic3, /fic2, /fic5, /fic2 ⁶

```
//Programa prueba_cache_tabla.c para testear las cachés de directorios > 1
//Sintaxis: prueba_cache_tabla <nombre_dispositivo> <texto>

#include "directorios.h"

int main(int argc, char **argv){

    if (argc!=3) {
        fprintf(stderr, RED "Sintaxis: ./prueba_cache_tabla <nombre_dispositivo> <texto>\n"
RESET);
        exit(FALLO);
    }

    // Montar dispositivo virtual
    bmount(argv[1]);

    //obtenemos el texto y su longitud
    char *buffer_texto = argv[2];
    int nbytes=strlen(buffer_texto);
    int offset=0;
    char rutas[5][6]={"/fic1", "/fic2", "/fic3", "/fic4", "/fic5"};
    char orden_acceso[12][6] = {"/fic2", "/fic3", "/fic2", "/fic1", "/fic5", "/fic2", "/fic4", "/fic5",
"/fic3", "/fic2", "/fic5", "/fic2"};

    // Crear ficheros
    for (int i=0; i<5; i++){
        mi_creat(rutas[i], 6);
        fprintf(stderr, MAGENTA "[prueba cache ()→ creado %s]\n" RESET, rutas[i]);
    }
}
```

⁶ Ejemplo similar al proporcionado en las diapositivas de teoría de gestión de memoria (paginación) de peticiones de páginas para los algoritmos clásicos de reemplazo.

```
//Escrituras
for (int j=0; j<12; j++){
    fprintf(stderr, CYAN "\n[prueba cache ()→ acceso a %s]\n" RESET, orden_acceso[j]);
    mi_write(orden_acceso[j], buffer_texto, offset, nbytes);
}

// Desmontar dispositivo virtual
bumount();
}
```

Ejemplo de ejecución para **FIFO** con **caché tamaño 3**:

```
[$ rm disco]
$ ./mi_mkfs disco 100000
$ ./prueba_cache_tabla disco "hola"
[prueba cache ()→ creado /fic1]
[prueba cache ()→ creado /fic2]
[prueba cache ()→ creado /fic3]
[prueba cache ()→ creado /fic4]
[prueba cache ()→ creado /fic5]

[prueba cache ()→ acceso a /fic2]
[mi_write() → Reemplazamos cache[0]: /fic2]

[prueba cache ()→ acceso a /fic3]
[mi_write() → Reemplazamos cache[1]: /fic3]

[prueba cache ()→ acceso a /fic2]
[mi_write() → Utilizamos cache[0]: /fic2]

[prueba cache ()→ acceso a /fic1]
[mi_write() → Reemplazamos cache[2]: /fic1]

[prueba cache ()→ acceso a /fic5]
[mi_write() → Reemplazamos cache[0]: /fic5]

[prueba cache ()→ acceso a /fic2]
[mi_write() → Reemplazamos cache[1]: /fic2]

[prueba cache ()→ acceso a /fic4]
[mi_write() → Reemplazamos cache[2]: /fic4]

[prueba cache ()→ acceso a /fic5]
[mi_write() → Utilizamos cache[0]: /fic5]
```

```
[prueba cache ()→ acceso a /fic3]
[mi_write() → Reemplazamos cache[0]: /fic3]

[prueba cache ()→ acceso a /fic2]
[mi_write() → Utilizamos cache[1]: /fic2]

[prueba cache ()→ acceso a /fic5]
[mi_write() → Reemplazamos cache[1]: /fic5]

[prueba cache ()→ acceso a /fic2]
[mi_write() → Reemplazamos cache[2]: /fic2]
```

Ejemplo de ejecución para LRU con caché tamaño 3:

```
$ ./mi_mkfs disco 100000
$ ./prueba_cache_tabla disco "hola"
[prueba cache ()→ creado /fic1]
[prueba cache ()→ creado /fic2]
[prueba cache ()→ creado /fic3]
[prueba cache ()→ creado /fic4]
[prueba cache ()→ creado /fic5]

[prueba cache ()→ acceso a /fic2]
[mi_write() → Reemplazamos cache[0]: /fic2]

[prueba cache ()→ acceso a /fic3]
[mi_write() → Reemplazamos cache[1]: /fic3]

[prueba cache ()→ acceso a /fic2]
[mi_write() → Utilizamos cache[0]: /fic2]

[prueba cache ()→ acceso a /fic1]
[mi_write() → Reemplazamos cache[2]: /fic1]

[prueba cache ()→ acceso a /fic5]
[mi_write() → Reemplazamos cache[1]: /fic5]

[prueba cache ()→ acceso a /fic2]
[mi_write() → Utilizamos cache[0]: /fic2]

[prueba cache ()→ acceso a /fic4]
[mi_write() → Reemplazamos cache[2]: /fic4]

[prueba cache ()→ acceso a /fic5]
[mi_write() → Utilizamos cache[1]: /fic5]
```

[prueba cache () → acceso a /fic3]
[mi_write() → Reemplazamos cache[0]: /fic3]

[prueba cache () → acceso a /fic2]
[mi_write() → Reemplazamos cache[2]: /fic2]

[prueba cache () → acceso a /fic5]
[mi_write() → Utilizamos cache[1]: /fic5]

[prueba cache () → acceso a /fic2]
[mi_write() → Utilizamos cache[2]: /fic2]